THE GUSTA | GUIA PASSO A PASSO

Etapa 1 — Atualizar somente os imports de pagamento.tsx

No início do arquivo, substitua os imports de React, Expo Router e React Native pelos equivalentes abaixo; preserve os imports dos estilos, FooterScreen e cores. Remova Linking, que deixará de ser usado.

```
import { useState, useCallback } from "react";
import { router, useFocusEffect } from "expo-router";
import { SafeAreaView } from "react-native-safe-area-context";
import {
  View, Text, ImageBackground, Image, TextInput,
  Pressable, ScrollView, Alert,
} from "react-native";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { API, IMAGEM } from "@/config/api";
import { ItemSacola, carregarSacola, limparSacola } from "@/config/sacola";
```

Abaixo desses imports, mantenha os seus imports existentes: globalStyle, pagamentoStyle, FooterScreen e cores.

Etapa 2 — Criar o tipo do endereço (antes do componente)

Acima de export default function PagamentoScreen(), adicione:

```
type Endereco = {
  id_endereco: number;
  nome_endereco: string;
  endereco: string;
  numero: string;
  complemento: string | null;
  bairro: string;
  cidade: string;
  uf: string;
  cep: string;
  principal_endereco: "SIM" | "NAO";
};
```

Etapa 3 — Substituir os dados fictícios pelo estado real

Dentro de PagamentoScreen(), exclua SOMENTE o array fixo const itensResumo = [...]. No mesmo lugar, coloque os estados e os cálculos a seguir:

```
const [itensResumo, setItensResumo] = useState<ItemSacola[]>([]);
const [enderecos, setEnderecos] = useState<Endereco[]>([]);
const [idEndereco, setIdEndereco] = useState<number | null>(null);
const [entrega, setEntrega] = useState<"SIM" | "NAO">("SIM");
const [formaPagamento, setFormaPagamento] = useState<"PIX" | "CARTAO" | "DINHEIRO">("PIX");
const [dataEntrega, setDataEntrega] = useState("");
const [horaEntrega, setHoraEntrega] = useState("");
const [observacao, setObservacao] = useState("");
const [erro, setErro] = useState("");
const [carregando, setCarregando] = useState(false);
const [enviando, setEnviando] = useState(false);
const enderecoSelecionado = enderecos.find(e => e.id_endereco === idEndereco);
const subtotal = itensResumo.reduce(
  (total, item) => total + item.valor_produto * item.quantidade, 0
);
const moeda = (valor: number) => valor.toLocaleString("pt-BR", {
  style: "currency", currency: "BRL",
});
const mascaraData = (valor: string) => {
  const numeros = valor.replace(/\D/g, "").slice(0, 8);

  return numeros
    .replace(/^(\d{2})(\d)/, "$1/$2")
    .replace(/^(\d{2})\/(\d{2})(\d)/, "$1/$2/$3");
```

```
};

const mascaraHora = (valor: string) => {
  const numeros = valor.replace(/\D/g, "").slice(0, 4);

  return numeros.replace(/^(\d{2})(\d)/, "$1:$2");
};
const converterDataHora = () => {
 const [dia, mes, ano] = dataEntrega.split("/");

 return `${ano}-${mes}-${dia} ${horaEntrega}`;
};
```

Etapa 4 — Carregar Sacola e endereços ao entrar na tela

Ainda dentro de PagamentoScreen(), após os estados e antes da função confirmarPedido, insira:

```
const carregarPagamento = useCallback(async () => {
  setCarregando(true);
  setErro("");
  try {
    const produtos = await carregarSacola();
    setItensResumo(produtos);
    const token = await AsyncStorage.getItem("token");
    if (!token) throw new Error("Sessão expirada. Faça login novamente.");
    const resposta = await fetch(`${API}/enderecos`, {
      headers: { Accept: "application/json", Authorization: `Bearer ${token}` },
    });
    const json = await resposta.json();
    if (!resposta.ok || !json.success) {
      throw new Error(json.message || "Não foi possível carregar endereços.");
    }
    const lista: Endereco[] = Array.isArray(json.data) ? json.data : [];
    setEnderecos(lista);
    setIdEndereco(atual =>
      lista.some(e => e.id_endereco === atual) ? atual :
      (lista.find(e => e.principal_endereco === "SIM")?.id_endereco
        ?? lista[0]?.id_endereco ?? null)
    );
  } catch (e) {
    setErro(e instanceof Error ? e.message : "Erro ao carregar pagamento.");
  } finally {
    setCarregando(false);
  }
}, []);

useFocusEffect(useCallback(() => { carregarPagamento(); }, [carregarPagamento]));
```

Observação: a API devolve json.data como array. O endereço principal vem primeiro. O carregamento por foco permite atualizar os endereços ao retornar de outra tela.

Etapa 5 — Tornar o endereço dinâmico SEM mover o bloco

No JSX, encontre o primeiro <View style={pagamentoStyle.endereco}> (antes de areaEntrega). Envolva-o com {entrega === "SIM" ? (...) : null}. Dentro do bloco, mantenha a imagem e os estilos, mas substitua somente os textos fixos por:

```
<Text style={pagamentoStyle.nomeEndereco}>
  {enderecoSelecionado?.nome_endereco ?? "Nenhum endereço cadastrado"}
</Text>
<Text style={pagamentoStyle.enderecoCompleto}>
  {enderecoSelecionado
    ? `${enderecoSelecionado.endereco},
${enderecoSelecionado.numero}${enderecoSelecionado.complemento ? ` -
${enderecoSelecionado.complemento}` : ""}`
    : "Cadastre um endereço para receber o pedido."}
</Text>
<Text style={pagamentoStyle.localCompleto}>
```

```
  {enderecoSelecionado
    ? `${enderecoSelecionado.bairro} - ${enderecoSelecionado.cidade} -
${enderecoSelecionado.uf}`
    : ""}
</Text>
```

Remova apenas o telefone fictício desse bloco: a resposta de GET /enderecos não contém telefone. Mantenha o Pressable Alterar na mesma posição; para selecionar entre endereços já cadastrados, substitua seu onPress vazio por uma abertura de Alert (exemplo abaixo).

```
onPress={() => {
  if (enderecos.length === 0) {
    Alert.alert("Endereço", "Nenhum endereço cadastrado.");
    return;
  }
  Alert.alert("Selecionar endereço", "Escolha um endereço:", [
    ...enderecos.map(e => ({
      text: `${e.nome_endereco} - ${e.endereco}, ${e.numero}`,
      onPress: () => setIdEndereco(e.id_endereco),
    })),
    { text: "Cancelar", style: "cancel" },
  ]);
}}
```

Atenção: no React Native Web, Alert.alert pode não funcionar em todas as plataformas. Para web, substitua depois por um modal próprio. Esta etapa pressupõe endereços já cadastrados; a tela de cadastro/edição de endereços é outro fluxo.

Etapa 6 — Fazer Entrega/Retirada funcionar na posição atual

No bloco pagamentoStyle.areaEntrega, mantenha os botões e substitua apenas suas aberturas e o título:

```
<Text style={pagamentoStyle.tituloEntrega}>Modalidade do pedido:</Text>

<Pressable style={pagamentoStyle.btnEntrega}
  onPress={() => setEntrega("SIM")}>
  <Text style={pagamentoStyle.txtEntrega}>
    {entrega === "SIM" ? "✓ Entrega" : "Entrega"}
  </Text>
</Pressable>
<Pressable style={pagamentoStyle.btnRetira}
  onPress={() => setEntrega("NAO")}>
  <Text style={pagamentoStyle.txtRetira}>
    {entrega === "NAO" ? "✓ Retirada" : "Retirada"}
  </Text>
</Pressable>
```

Retire o texto fixo "40 - 60 min"; a API não informa essa previsão. O endereço fica oculto para retirada e id_endereco não será enviado nesse caso.

Etapa 7 — Adicionar data e horário (único bloco visual novo)

Depois de fechar pagamentoStyle.areaEntrega e ANTES de pagamentoStyle.areaPagamento, acrescente o bloco abaixo. O usuário informa o formato exigido pela API (AAAA-MM-DD HH:MM).

```
<View style={pagamentoStyle.areaObservacao}>
  <View style={pagamentoStyle.camposDataHora}>
    <Text style={pagamentoStyle.tituloObservacao}>
      Data da {entrega === "SIM" ? "entrega" : "retirada"}
    </Text>
    <TextInput
      style={pagamentoStyle.txtDataHora}
      placeholder="DD/MM/AAAA"
```

```
        keyboardType="numeric"
        placeholderTextColor={cores.cinza}
        value={dataEntrega}
        onChangeText={(texto) => setDataEntrega(mascaraData(texto))}
        maxLength={10}
      />
    </View>
    <View style={pagamentoStyle.camposDataHora}>
      <Text style={pagamentoStyle.tituloObservacao}>
        Horário da {entrega === "SIM" ? "entrega" : "retirada"}
      </Text>
      <TextInput
        style={pagamentoStyle.txtDataHora}
        placeholder="HH:MM"
        keyboardType="numeric"
        placeholderTextColor={cores.cinza}
        value={horaEntrega}
        onChangeText={(texto) => setHoraEntrega(mascaraHora(texto))}
        maxLength={5}
      />
    </View>
  </View>
```

A validação de data real e futura será feita antes do POST. Um seletor de data nativo pode substituir o TextInput em etapa futura.

Etapa 8 — Ativar as formas de pagamento

No bloco pagamentoStyle.areaPagamento, preserve a ordem PIX → Cartão → Dinheiro e adicione os onPress nos três Pressable existentes:

```
<Pressable style={pagamentoStyle.btnPix}
  onPress={() => setFormaPagamento("PIX")}>
  <Text style={pagamentoStyle.txtPix}>
    {formaPagamento === "PIX" ? "✓ Pix" : "Pix"}
  </Text>
</Pressable>
<Pressable style={pagamentoStyle.btnCartao}
  onPress={() => setFormaPagamento("CARTAO")}>
  <Text style={pagamentoStyle.txtCartao}>
    {formaPagamento === "CARTAO" ? "✓ Cartão" : "Cartão"}
  </Text>
</Pressable>
<Pressable style={pagamentoStyle.btnDinheiro}
  onPress={() => setFormaPagamento("DINHEIRO")}>
  <Text style={pagamentoStyle.txtDinheiro}>
    {formaPagamento === "DINHEIRO" ? "✓ Dinheiro" : "Dinheiro"}
  </Text>
</Pressable>
```

Etapa 9 — Ligar o campo de observação ao estado

No TextInput que já existe dentro de pagamentoStyle.areaObservacao, adicione somente as propriedades abaixo, mantendo todos os estilos:

```
value={observacao}
onChangeText={setObservacao}
maxLength={500}
multiline
```

Etapa 10 — Trocar o resumo fictício por produtos da Sacola

No bloco pagamentoStyle.resumo, substitua APENAS o conteúdo de itensResumo.map pelo código abaixo. Não mude a posição do resumo.

```
{itensResumo.map((item) => (
  <View style={pagamentoStyle.itemResumo} key={item.id_produto}>
    <Image
      style={pagamentoStyle.imgProduto}
      source={{ uri: item.foto_produto
        ? `${IMAGEM}/${item.foto_produto}`
        : `${IMAGEM}/produto/sem-imagem.png` }}
      resizeMode="stretch"
    />
    <Text style={pagamentoStyle.nomeProduto}>
      {item.quantidade}x {item.nome_produto}
    </Text>
    <Text style={pagamentoStyle.valorProduto}>
      {moeda(item.valor_produto * item.quantidade)}
    </Text>
  </View>
))}
```

Ainda dentro de pagamentoStyle.subtotal: substitua o subtotal fixo por {moeda(subtotal)}. Remova os blocos areaValorEntrega e areaDesconto com valores fictícios (R$ 8,00, THEGUSTA40 e R$ -15,84). Em areaTotal, exiba {moeda(subtotal)} como valor provisório, sem frete nem desconto.

Acima do botão Confirmar pedido, acrescente um aviso:

```
<Text>Valor dos produtos: {moeda(subtotal)}. O total definitivo
será calculado pela API ao registrar o pedido.</Text>
{carregando ? <Text>Carregando informações...</Text> : null}
{erro ? <Text style={{ color: "#B00020" }}>{erro}</Text> : null}
```

Etapa 11 — Substituir confirmarPedido (WhatsApp) pelo POST /pedidos

Apague somente a função confirmarPedido antiga (que monta texto para WhatsApp e usa Linking). No mesmo lugar, insira as funções a seguir. Não altere o JSX de layout.

```
function dataValidaFutura(texto: string): boolean {
  const m = /^(\d{4})-(\d{2})-(\d{2}) (\d{2}):(\d{2})$/.exec(texto);
  if (!m) return false;
  const [ano, mes, dia, hora, minuto] = m.slice(1).map(Number);
  const data = new Date(ano, mes - 1, dia, hora, minuto);
  return data.getFullYear() === ano && data.getMonth() === mes - 1 &&
    data.getDate() === dia && data.getHours() === hora &&
    data.getMinutes() === minuto && data.getTime() > Date.now();
}

async function confirmarPedido() {
  if (enviando) return;
  if (itensResumo.length === 0) { setErro("A sacola está vazia."); return; }
  if (entrega === "SIM" && !idEndereco) {
    setErro("Selecione um endereço de entrega."); return;
  }
  const dataHoraPedido = converterDataHora();

  if (!dataValidaFutura(dataHoraPedido)) {
    setErro("Informe uma data e horário futuros válidos.");
    return;
  }
  setEnviando(true);
  setErro("");
  try {
    const token = await AsyncStorage.getItem("token");
    if (!token) throw new Error("Sessão expirada. Faça login novamente.");
    const corpo = {
      itens: itensResumo.map(item => ({
        id_produto: item.id_produto, quantidade: item.quantidade,
      })),
      entrega,
      ...(entrega === "SIM" ? { id_endereco: idEndereco } : {}),
```

```
      forma_pagamento: formaPagamento,
      data_entrega: dataHoraPedido,
      ...(observacao.trim() ? { observacao: observacao.trim() } : {}),
    };
    const resposta = await fetch(`${API}/pedidos`, {
      method: "POST",
      headers: {
        Accept: "application/json",
        "Content-Type": "application/json",
        Authorization: `Bearer ${token}`,
      },
      body: JSON.stringify(corpo),
    });
    const json = await resposta.json();
    if (!resposta.ok || !json.success) {
      throw new Error(json.message || "Não foi possível criar o pedido.");
    }

    // A partir daqui o pedido JÁ EXISTE na API.
    const pedido = json.data;
    const numeroPedido = pedido.id_venda;
    const listaProdutos = itensResumo.map(item =>
      `${item.quantidade}x ${item.nome_produto} - ` +
      moeda(item.valor_produto * item.quantidade)
    ).join("\n");
    const local = entrega === "SIM" && enderecoSelecionado
      ? `Entrega: ${enderecoSelecionado.nome_endereco} - ` +
        `${enderecoSelecionado.endereco}, ${enderecoSelecionado.numero}`
      : "Retirada no estabelecimento";
    const valorFinal = Number(pedido.valor_venda);
    const valorConfirmado = Number.isFinite(valorFinal)
      ? moeda(valorFinal)
      : "Consulte o valor no pedido registrado";
    const mensagem = [
      "🍰 *NOVO PEDIDO - THE GUSTA*",
      `*Pedido nº ${numeroPedido}*`,
      "", "📋 *Itens do pedido:*", listaProdutos,
      "", `Subtotal (prévia): ${moeda(subtotal)}`,
      `💰 *Total confirmado pela API:* ${valorConfirmado}`,
      "", `📍 ${local}`,
      `📅 Data desejada: ${dataEntrega} às ${horaEntrega}`,
      `💳 Forma de pagamento: ${formaPagamento}`,
      ...(observacao.trim() ? [`📝 ${observacao.trim()}`] : []),
    ].join("\n");

    try {
      await limparSacola();
    } catch (erroLimpeza) {
      console.warn("Pedido criado; falha ao limpar sacola:", erroLimpeza);
    }
    // Mantenha o telefone usado na sua função original.
    const telefone = "5511964440092";
    const url = `https://wa.me/${telefone}?text=${encodeURIComponent(mensagem)}`;
    try {
      await Linking.openURL(url);
    } catch (erroWhatsApp) {
      console.warn("WhatsApp não abriu:", erroWhatsApp);
      setErro(`Pedido nº ${numeroPedido} registrado! ` +
        "Não foi possível abrir o WhatsApp. Consulte Meus Pedidos.");
    }
    // Não criar outro pedido se o WhatsApp falhar.
    // Não navegar imediatamente: isso pode interromper o WhatsApp.
  } catch (e) {
    setErro(e instanceof Error ? e.message : "Erro ao confirmar pedido.");
  } finally {
    setEnviando(false);
  }
}
```

Importante: ajuste router.replace("/meus-pedidos") para a rota REAL existente em src/app. O Alert.alert pode não aparecer na Web; se necessário, apresente a mensagem na tela antes de navegar. Um erro de conexão depois do envio pode ter criado o pedido; antes de reenviar, consulte o histórico para evitar duplicidade.

Etapa 12 — Evitar duplo clique no botão

No Pressable pagamentoStyle.btnPedido já existente, mantenha o estilo e substitua sua abertura e texto:

```
<Pressable
  style={pagamentoStyle.btnPedido}
  disabled={enviando || carregando || itensResumo.length === 0}
  onPress={confirmarPedido}
>
  <Text style={pagamentoStyle.txtPedido}>
    {enviando ? "Enviando pedido..." : "Confirmar pedido"}
  </Text>
</Pressable>
```